# An Executable Safety Ladder Toward Unbounded Viewstamped Replication

The uVRR research project

Research checkpoint — 6 September 2026

## Abstract

Unbounded Viewstamped Replication Revisited (uVRR) seeks to combine replicated volatile state, low-delay primary replacement, and reconfiguration without imposing a fixed pipeline bound. These requirements cross several distinct proof boundaries. We audit an inherited Lean development of Turner's era-indexed Paxos agreement argument and extend it with a finite-support weighted-majority theorem, a derivation of the proposal-era bound from suffix promises, operational acceptor invariants, and fault-injection controls. Every positive result is checked by Lean 4.33.1 using its core libraries. Executable Showboat documents preserve source, build commands, and theorem axiom reports. All eight inherited transcripts reproduce, although the eighth records an unsuccessful proof attempt. We distinguish this reproducibility result from mathematical correctness and from protocol refinement. We also separate counterexamples establishing independence of assumptions from claims of universal necessity. A section-by-section examination of Viewstamped Replication Revisited identifies the outstanding integration obligations: whole-log view selection, recovery fencing, repeated reconfiguration, client semantics, and progress. Bounded Leanstral experiments are reported as part of the methodology, including unsuccessful runs and unmeasured costs. This checkpoint establishes reusable verified components; an end-to-end uVRR correctness theorem and a millisecond failover measurement remain open.

## 1 Problem and contribution

The motivating uVRR proposal asks whether a strongly consistent replicated service can replace failed members while continuing to process requests, without forcing its log to disk on the normal agreement path [3]. A precise answer requires more than an intersection calculation. The membership used by a vote must be agreed; a replacement primary must preserve committed operations; a crashed replica must not cast votes using forgotten state; and a reply must correspond to an operation with the promised client semantics. Progress and elapsed time impose additional assumptions.

Turner's UPaxos construction permits a sequence of consensus instances to remain unbounded during suitable reconfigurations [1]. Its safety argument provides a natural starting point. The inherited development encoded the Synod history invariants, reduced an era-indexed history to them, checked two finite reconfiguration schedules, and isolated a leader's casting-vote schedule. It explicitly excluded a proof of VRR itself. Our audit confirms the compiled mathematical results while narrowing several statements in the associated report.

This checkpoint adds three positive results. First, an arbitrary finite-support version of UPaxos's weighted intersection lemma replaces dependence on a four-node example. Second, the omitted suffix-promise and known-instance frontier conditions now derive the proposal-era inequality used in the agreement reduction. Third, induction over an acceptor transition system establishes the historical promise conditions for that component. Constructive negative controls test the role of selected assumptions. The laboratory book records intermediate failures, tool outcomes, and restart instructions.

The central unresolved claim is stated explicitly: *every execution of the intended uVRR message protocol, under a specified crash-recovery model, implements a linearizable replicated state machine across arbitrarily many reconfigurations.* The results below discharge parts of this claim. They do not discharge it by renaming a Paxos ballot a VRR view.

## 2 Setting and trust boundary

Let $A$, $B$, and $V$ be node, ballot, and value types. A node set is a predicate $A \to \mathrm{Prop}$; a quorum family is a predicate on such sets. Define

$$Q \frown R \quad \Longleftrightarrow \quad \forall q \in Q,\ r \in R, \quad q \cap r \neq \varnothing.$$

This relation is symmetric. It is a property of two families, not a requirement that every pair of members within either family intersect. Empty families make some such statements vacuous. Consequently nonemptiness conditions in the agreement theorem must remain visible.

The history model records messages that have ever been emitted. Its predicates include free promises $p(a, b)$, promises carrying a last vote $p(a, b; c)$, proposals, acceptances, and choices. A function $v(b)$ gives the unique value associated with each ballot. The representation presupposes ballot ownership or an equivalent mechanism preventing two values at one ballot;

it is not a proof that a wire protocol enforces that mechanism.

The ballot order is well founded and strictly total. The era development uses $B = \mathbb{N} \times \mathbb{N}$ ordered lexicographically, so that unboundedly many rounds are available in each era. This is a mathematical domain; a fixed-width implementation needs a separate overflow argument.

The executable library imports Lean core and other modules in the same development. It has no Mathlib dependency. A proof may use standard Lean axioms `propext`, `Quot.sound`, and `Classical.choice`; these are distinguished from additional protocol hypotheses appearing in theorem types. Printing axioms alone does not expose every protocol assumption: a conditional theorem can have an empty axiom footprint and strong hypotheses. The central generic Synod theorem has no axiom dependencies. Other results use subsets of the three standard axioms.

# 3 Conditional agreement and reconfiguration

## 3.1 The Synod contract

The inherited structure `Synod.Inv` expresses six conditions:

1. If $b_2 < b_1$, $b_1$ is proposed, and $b_2$ is chosen, then $\mathcal{Q}^{\mathrm{I}}(b_1) \frown \mathcal{Q}^{\mathrm{II}}(b_2)$.
2. A free promise at $b$ excludes every acceptance below $b$.
3. A report $p(a, b; c)$ names an accepted ballot $c < b$ and excludes acceptances strictly between $c$ and $b$.
4. A proposal has a phase-I quorum of replies. If any report carries a vote, the proposed value equals that of a maximal reported ballot.
5. Every acceptance has a proposal.
6. Every choice has a phase-II quorum of acceptances.

The maximal reported ballot in condition 4 is supplied existentially. An implementation must obtain it from a finite collection and enforce the value rule. The history theorem does not construct a leader's message handler.

**Theorem 1** (Synod agreement)**.** *Assume the six conditions, a well-founded strict total ballot order, and nonempty phase-II quorums. If $b$ and $c$ are chosen, then $v(b) = v(c)$.*

*Proof.* For a chosen $c$ and a proposed $b > c$, intersect the decision quorum at $c$ with the promise quorum at $b$. A common acceptor cannot have supplied a free promise at $b$, since it accepted $c$. Its reported vote is at least $c$ by condition 3. The greatest reported ballot $m$ therefore satisfies $c \le m < b$. If $m = c$, condition 4 gives the conclusion. Otherwise apply well-founded induction to the proposal at $m$, which exists by conditions 3 and 5. Finally, a chosen ballot has a proposal by conditions 5 and 6 and decision-quorum nonemptiness. Trichotomy reduces agreement to the preceding argument or equality of ballots. □

The checked declaration is `Synod.theorem8`; its intermediate induction lemma is `Synod.lemma7`. This proof follows UPaxos Appendix B.

## 3.2 Era-indexed families

Let $e(b)$ be a ballot's era and $e(i)$ the era of instance $i$. Phase I uses $\mathcal{Q}^{\mathrm{I}}_{e(b)}$ and phase II uses $\mathcal{Q}^{\mathrm{II}}_{e(i)}$. The configuration condition is

$$\forall e, \qquad \mathcal{Q}^{\mathrm{II}}_{e} \frown \mathcal{Q}^{\mathrm{I}}_{e} \quad \wedge \quad \mathcal{Q}^{\mathrm{I}}_{e} \frown \mathcal{Q}^{\mathrm{II}}_{e+1}. \tag{1}$$

The reduction also requires proposal and choice era bounds:

$$\mathrm{proposed}_i(b) \Longrightarrow e(b) \le e(i), \tag{2}$$

$$\mathrm{chosen}_i(b) \Longrightarrow e(i) \le e(b) + 1. \tag{3}$$

Ballot order must respect era order. These conditions are separate from Equation (1); the latter alone is not a complete protocol contract.

**Lemma 2** (Relevant intersection)**.** *If $b_2 < b_1$, $b_1$ is proposed at $i$, and $b_2$ is chosen at $i$, then $\mathcal{Q}^{\mathrm{I}}_{e(b_1)} \frown \mathcal{Q}^{\mathrm{II}}_{e(i)}$.*

*Proof.* The bounds give $e(b_1) \le e(i) \le e(b_2) + 1 \le e(b_1) + 1$. Thus $e(i)$ is either $e(b_1)$ or $e(b_1) + 1$. The first case uses the intra-era half of Equation (1), by symmetry; the second uses its cross-era half. □

Instantiating the Synod theorem for every instance yields `Paxos.theorem10`. Its type also contains the promise, value-selection, acceptance, and choice conditions. Agreement holds for every history satisfying them, across any number of eras. This quantification does not establish that an operational sequence of membership changes preserves them.

## 3.3 Deriving the proposal-era bound

The inherited `EraLe` hypothesis, Equation (2), bypassed part of UPaxos Figure 2. The extension `MultiPromise.History` restores a known-instance frontier $i_{\max}$ and suffix promises $p_{\ge j}(a, b)$. A suffix certificate expands to a free promise at every $i \ge j$.

**Theorem 3** (Suffix-promise reduction)**.** *Suppose instance eras are nondecreasing. A promise at $j$, including a suffix promise, satisfies $e(b) \le e(\min(j, i_{\max}))$. Each proposal at $i$ has a nonempty phase-I quorum whose members supply a free promise at $i$, a last-vote report at $i$, or a suffix promise starting at some $j \le i$. Then* $\mathrm{proposed}_i(b) \Rightarrow e(b) \le e(i)$.

*Proof.* Choose a member of the proposal quorum. For a point promise, $\min(i, i_{\max}) \le i$. For a suffix promise, $\min(j, i_{\max}) \le j \le i$. In either case monotonicity of instance eras transports the promise's era bound to $e(i)$. □

`MultiPromise.History.expanded_inv` additionally transports the full promise history to the inherited invariant structure. The resulting `agreement` theorem retains Figure 2's known-frontier bound for choices. The expansion is an invariant reduction; the append-only evolution of configuration knowledge and the creation of suffix certificates still need an operational proof. A single certificate can denote every slot $j + d$ for $d \in \mathbb{N}$, as checked by `suffix_unbounded`. This proves representational unboundedness, not completion of those slots.

# 4 General weighted intersection

For a finite support list $L$ and natural weights $w$, let $W = \sum_{a \in L} w(a)$ and $W_q = \sum_{a \in L \cap q} w(a)$. Define $M_L(w) = \{q : 2W_q > W\}$. A duplicate-free list gives the usual finite-set interpretation; repeated entries instead contribute repeated weight. No bound on the cardinality of $L$ is fixed in the theorem.

**Lemma 4** (Disjoint selection bound)**.** *For disjoint $q, r$ and natural weights $x, y$,*

$$2X_q + 2Y_r \le X + Y + \sum_{a \in L} |x(a) - y(a)|.$$

*Proof.* At each node, at most one of the two selections contributes. The required pointwise inequality follows from $2x \le x + y + |x - y|$ or its symmetric form; if neither selects the node it is immediate. Sum by induction over $L$. The Lean encoding uses $(x-y)+(y-x)$ with natural truncated subtraction. □

**Theorem 5** (Scaled weighted-majority overlap)**.** *For positive integers $k, l$, if*

$$D = \sum_{a \in L} |kw(a) - lw'(a)| \le 1,$$

*then $M_L(w) \frown M_L(w')$.*

*Proof.* Positive rescaling preserves strict majority. If two majorities were disjoint, their scaled sums would exceed their respective scaled totals by at least one each, since all sums are integers. Their combined excess is at least two, contradicting Lemma 4 and $D \le 1$. □

This is UPaxos Appendix A's Lemma 2, proved here by a pointwise discrepancy bound. `WeightedGeneral.scaled_overlap` is the checked declaration. The unit-change result, equal-rescaling result, and self-intersection follow as corollaries. Zero total weight yields an empty majority family, so self-intersection by itself does not assert availability.

The uniform bound is sharp. With two nodes, weights $(1, 0)$ and $(0, 1)$ have distance two and admit disjoint singleton majorities. This is checked by `distance_two_counterexample`. Some distance-two changes nevertheless preserve overlap; for example $(2, 2, 2, 0)$ to $(4, 2, 2, 0)$ does. Thus sharpness means that a larger universal allowance would admit counterexamples, not that all changes beyond the allowance are unsafe.

The inherited four-node weighted schedule remains an independent executable example. Its seven rows obey the required overlaps. It is useful for a concrete deployment shape; it is no longer the only evidence for the general weighted construction.

# 5 Operational promises and fault controls

## 5.1 A checked acceptor transition system

An acceptor state is $(f, H, R)$: a promise watermark $f \in \mathbb{N}$, a newest-first vote history $H$, and a reply history $R$. Initially these are $(0, [], [])$. Acceptance requires $f \le b$; a promise requires $f < b$. The transitions are:

$$\begin{aligned} \mathrm{accept}_b(f, H, R) &= (b, b :: H, R), \\ \mathrm{promise}_b(f, H, R) &= (b, H, (b, \mathrm{head?}\ H) :: R). \end{aligned}$$

An empty head produces a free reply; otherwise the reply carries the latest accepted ballot. The histories are ghost evidence for the proof. This module does not prescribe retaining an unbounded physical list at a replica.

**Theorem 6** (Historical promise safety)**.** *In every state reachable by a finite execution, a free reply at $b$ excludes votes below $b$. A reply carrying $c$ at $b$ establishes $c < b$, an acceptance of $c$, and absence of acceptances strictly between $c$ and $b$.*

*Proof.* Strengthen the induction invariant: all votes are at most the watermark; the head vote dominates its tail; every emitted reply's ballot is at most the watermark; and each reply satisfies the stated historical conditions. An acceptance preserves every old reply because its ballot is at least the old watermark. A promise reads the maximal vote and raises the watermark above it. Neither transition removes prior evidence. Induction on reachable states proves the claim. □

The declarations are `Acceptor.reachable_inv`, `free_forbids`, and `report_last`. They discharge S2 and S3 for a single-era acceptor. They do not establish the proposal value rule, an entire log's view-change selection, or restoration after volatile-state loss. Extending the component to era-tagged views also remains an explicit integration task.

### 5.2 What the negative controls establish

The controls separate two questions: whether the proof notices an altered definition, and whether removing an assumption admits an inconsistent history. The first is measured by compiler rejection; the second by constructing and checking a counterexample with the remaining conditions intact.

- *Cross-era overlap.* The inherited two-node history satisfies intra-era overlap and the other encoded era-history conditions, but chooses both Boolean values at one slot when the cross overlap is absent.
- *Value selection.* One acceptor reports ballot 0 at ballot 1, yet the two proposals use different values. All listed Synod conditions except S4 hold and both ballots are chosen.
- *Promise fencing.* One acceptor supplies free promises at two rounds despite its earlier vote. All listed conditions except S2 hold, and agreement fails.
- *Nonempty decisions.* Empty phase-II quorums permit choices with no proposal or acceptance. The six history invariants hold vacuously where appropriate, and the two selected values differ.
- *Operational guard.* After a free promise at round 2, accepting round 1 without checking the watermark violates the reachable invariant.

These examples establish independence within the specified contracts. They do not prove a universal necessity theorem over all consensus algorithms, nor do they enumerate every combination of omitted assumptions.

Separately, the mutation harness replaces the checker's universal quantifier over quorums by an existential one, and replaces a safe concrete quorum family by an unsafe family. Lean rejects each altered source while accepting the unchanged control. An injected `sorry` demonstrates a different failure mode: compilation succeeds but `#print axioms` exposes `sorryAx`. The harness bounds child memory, work and wall time and modifies only temporary copies. Infrastructure failures are not counted as killed mutants.

## 6 The boundary between UPaxos and VRR

VRR-2012 is a replication protocol, with client handling and recovery in addition to the agreement mechanism [2]. Its normal path requires ordered log preparation and a quorum of acknowledgements before execution. View change selects the log with the greatest last-normal view, breaking ties by length. That whole-log selection rule needs a proof relating logs from different views; a per-slot maximal-vote rule is not automatically the same operation.

The distinction becomes essential after crashes. A recovering replica is excluded from normal processing and view changes. Its recovery exchange requires responses from distinct peers, a fresh nonce, and state from the primary of the greatest reported view. The preceding round of StartViewChange messages provides replicated knowledge of view advancement. The paper's Section 8.2 gives a failure scenario when that round is removed. Replacing volatile state by an acceptor watermark that never disappears would bypass precisely this obligation.

The 2012 reconfiguration protocol stops taking new client requests in the old epoch, transfers the committed state, and activates the replacement group. Section 7.5 explicitly discusses the resulting delay and prewarming as a way to reduce it. The uVRR objective instead calls for progress during the transition. Equation (1) and casting-vote scheduling address part of that difference, while the interaction with recovery remains to be proved.

The inherited casting-vote lemma assumes an old decision quorum $q$, a new promise quorum $q'$, and a leader $\ell$ with $q \cap q' = \{\ell\}$. Sending new promises only to $q' \setminus \{\ell\}$ leaves the old acceptance guard unaffected at the other members of $q$. Once $\ell$ promises, the new quorum is complete. This is a schedule-dependent noninterference argument. Availability of those quorums, continued processing by the leader, successful message delivery, and the ability to repeat the procedure require additional proofs. The lemma includes no clock or duration.

The repository's `VrrCoreEras.tla` is explicitly a design model for a single era transition, rather than a statement that the Rust implementation already refines it. Its gate checks both cross-era view/commit directions and fence/recovery intersections. It also records multiple live recovery nonces. The Lean history theorem's narrower directional overlap must not be used to remove those checks without proving the relevant transition refinement. Table 1 records the outstanding source-level obligations, including optional VRR optimizations whose semantics differ from the base protocol.

## 7 Reproducibility and research methodology

### 7.1 Executable evidence

Each positive rung contains the entire module text, a build invocation, and axiom reports for its stated results. `showboat verify` reruns the commands and compares their captured output [4]. All eight inherited documents pass this check. The eighth contains the expected compiler failure of a historical Leanstral draft, so its successful replay does not establish the missing weighted theorem. Rung 9 supplies that theorem.

The checked additions are numbered 9 (general weights), 10 (operational acceptor), 11 (suffix

Table 1: Coverage of the whole VRR-2012 paper at this checkpoint. "Open" means no end-to-end Lean result is claimed for that obligation.

| Source section | Required obligation | Checked evidence and present boundary |
|---|---|---|
| 1–3: model and architecture | Crash-only failures, authenticated identities, network model, deterministic execution, replica thresholds | Explicit assumptions; quorum intersection is checked. No general environmental refinement theorem. |
| 4.1: normal operation | One value per view/slot, ordered preparation, distinct quorum support, commit before execution, request deduplication | Conditional slot agreement; operational acceptor promise rules. Proposer, clients, and execution integration open. |
| 4.2: view changes | Sender status, two message rounds, latest-normal-view/length selection, preservation of every committed prefix | Promise fencing component checked. Whole-log selection and all message transitions open. |
| 4.3: replica recovery | Exclusion until fresh quorum evidence and current-primary state; restored votes consistent with prior activity | Recovery and its interaction with fencing open. |
| 4.4–4.5: applications and clients | Chosen nondeterministic inputs; monotone request identifiers across client crashes | Explicit application/client refinement obligations, open. |
| 5.1–5.3: practical transfer | Checkpoint fidelity, suffix completeness, garbage collection, view-sensitive truncation, short view-change messages | Open; source paper's full-log argument does not validate optimized transfer automatically. |
| 6.1–6.2: witnesses and batching | Witness participation roles; batch semantics and operation order | Optional refinements open. No correctness claim inherited merely from batching. |
| 6.3: fast reads | Lease exclusion and clock assumptions, or the weaker semantics of backup reads | Open; the backup-read variant is not claimed linearizable by the source. |
| 7.1–7.5: reconfiguration | Epoch establishment, transfer completion, safe retirement, duplicate messages, client routing | Era-history reduction and quorum calculations checked; operational repeated uVRR reconfiguration open. |
| 8.1–8.3: correctness arguments | Compose normal operation, recovery and reconfiguration; state progress assumptions | Component safety results checked. Composition and liveness open. |
| 9: conclusion | Relate the completed results to the replication service | No additional theorem is inferred from the source's summary. |

promises), and 12 (fault controls). The original report is retained as an audit input. Its statement that every rung includes a successful build and axiom report does not hold for rung 8.

The independent baseline audit ran the unmodified library build and every original transcript, inspected theorem types and axiom reports, and preserved input hashes. Subsequent changes were checked in small increments. The library build and transcript checks complement the Rust format, lint, test and documentation gates. Passing the Rust suite does not establish a formal refinement to Lean. A vendor-directory match in an inherited repository text gate is documented rather than suppressed by modifying unrelated code.

An independent TLC 2.19 breadth-first run of `VrrCoreEras.cfg` completed in 137.67 seconds, generating 2,743,933 states and finding 837,204 distinct states, with no error reported and an empty remaining queue. Its scope is the three-node `inc3` scenario, one command value, maximum log length three, two eras, and view index zero. `MaxEpoch=0` disables crashes; this run supplies no crash-recovery coverage. TLC reported a fingerprint collision probability estimate of $2.2 \times 10^{-7}$ from the observed fingerprints. This finite exploration is separate from kernel-checked unbounded theorems. The `VrrCoreErasM1.cfg` transfer mutation produced the expected `FrontiersOrdered` violation in 1.2 seconds. Source and configuration hashes and full logs are retained for both runs.

## 7.2 Bounded multi-model proof assistance

Leanstral is a Lean-oriented model available through Mistral's tooling. Its release article reports benchmarks and case studies [5]; those results do not establish cost effectiveness for this core-only development. Here the model is a source of candidate proof text. The local Lean kernel, the theorem type, and an explicit axiom policy determine acceptance.

The inherited record reports a failed attempt on the general weighted lemma, including a large Lean process. We preserved that record and replayed the draft's compiler failure. We did not independently reproduce its historical peak memory or elapsed time.

The new experiment isolated a single list-induction obligation with unchanged definitions and statement. The first Vibe run had a 12-turn, 28,000-token, 180-second and 2-GiB sampled process-tree memory limit, with a configured USD 2 cap. It stopped after 8.19 seconds when scaffold usage exceeded the token limit, before a proof edit. The second used 6 turns, 150,000 tokens, 120 seconds and a configured USD 0.50 cap. It

stopped after 76.39 seconds on the cumulative-token limit, leaving the original proof hole. Peak sampled resident memory was about 205 MiB in each run. Neither cap is a measurement of actual billing, and sampling is not an operating-system memory guarantee. Two direct-API routing probes also failed. The weighted theorem was then completed and compiled locally.

No accepted Leanstral-generated proof is attributed to these runs. Their methodological result is that scaffold overhead dominated a tiny proof target; repeating the same configuration was not justified. Future comparisons should record exact prompts, tool context, candidate edits, compiler iterations, token usage where available, elapsed time, and invoiced cost. A claim of multi-model efficiency needs comparable successful obligations and a baseline workflow. This experiment supplies neither a measured dollar saving nor a general negative judgment about the model.

The laboratory book distinguishes user-supplied authorship of earlier work from independently checked artifacts. This permits credit to prior model assistance without making model identity part of the trusted proof base.

# 8 Next proof boundary and performance claims

The next integration theorem must show that the intended VRR view-change transition selects a log containing every previously committed entry, despite divergent uncommitted suffixes and partially completed prior views. A recovery proof must then justify returning a replica to participation using fresh evidence, including the fence knowledge created before a crash. Finally, reconfiguration must preserve these facts for an arbitrary sequence of membership changes and state transfers. These are concrete proof obligations, not consequences of quorum intersection alone.

The term *non-stop* also needs an operational definition. A useful conditional progress statement would quantify over an available casting-vote schedule and fair delivery while allowing arbitrary pipeline length. It would state which classes of requests keep completing while the new phase-I exchange is pending. It would not promise service after loss of all necessary quorums. Similarly, no fixed failover time follows in an asynchronous model allowing unbounded message delay. A single-digit-millisecond result requires specified failure detection, network and workload conditions, an implementation, and measured latency distributions.

The present results establish exact component theorems with executable evidence and targeted counterexamples. Completing the operational composition will determine the scope of the final uVRR correctness claim. Performance and novelty claims will be evaluated against that completed specification and its experimental evidence.

# A Reproduction commands

From the repository root, with Lean and Showboat installed:

```
export PATH="$HOME/.elan/bin:$PATH"
cd formal/uvrr-lean
lake build
for f in ladder/[0-9][0-9]-*.md; do
  showboat verify "$f" || exit
done
python3 check_mutations.py
python3 check_axioms.py
```

The broader glob includes new two-digit rungs; `0*.md` only selects numbers below 10. These checks use the pinned Lean toolchain. Showboat transcripts depend on command output as well as exit status; a fresh build is run independently of transcript replay. Historical rejected drafts are quarantined outside the compiled library.

# References


[1] D. C. Turner, "Unbounded Pipelining in Dynamically Reconfigurable Paxos Clusters," revision 1A9DBA37, 14 August 2017. https://tessanddave.com/paxos-reconf-latest.pdf.

[2] B. Liskov and J. Cowling, "Viewstamped Replication Revisited," MIT-CSAIL-TR-2012-021, 23 July 2012. https://hdl.handle.net/1721.1/71763.

[3] simbo1905, "Unbounded Viewstamped Replication Revisited?" 12 August 2026. https://simbo1905.wordpress.com/2026/08/12/viewstamped-replication-revisited/.

[4] S. Willison, "Showboat: Create executable demo documents," software, version 0.6.1 used for the inherited evidence. https://github.com/simonw/showboat.

[5] Mistral AI, "Leanstral: Open-Source foundation for trustworthy vibe-coding," 16 March 2026, accessed 6 September 2026. https://mistral.ai/news/leanstral/.